ОТЧЁТ ОБ ОЦЕНКЕ ЗАЩИЩЁННОСТИ

Анализ защищённости веб-ресурса

https://stk-96.ru

Пассивная и слабо-активная разведка. Проверка конфигурации, анализ открытых интерфейсов, идентификация компонентов CMS.

Объект исследования:	**https://stk-96.ru**
Тип работ:	Внешний аудит, black-box
Дата проведения:	13 октября 2026 г.
Исполнитель:	Специалист по информационной безопасности
Методы:	Ручной анализ, PowerShell 5.1, HTTP(S)-запросы
Ограничения:	Без эксплуатации уязвимостей и нагрузки на ресурс

Резюме для руководства

> В ходе работ подтверждена **публично доступная утечка журнала отладки WordPress**, содержащего полные серверные пути и внутренние имена файлов CMS. Дополнительно выявлены: устаревшая версия интерпретатора PHP, отсутствие базовых защитных HTTP-заголовков, раскрытие логина администратора через REST API и отсутствие двухфакторной аутентификации на форме входа.
>
> Критических уязвимостей класса RCE, SQL-инъекций, обхода аутентификации в рамках согласованного объёма не обнаружено.

Сводная таблица находок

№	Уровень	Находка	Адрес
1	CRITICAL	Публично доступный журнал отладки WordPress	`/wp-content/debug.log`
2	HIGH	Устаревшая версия PHP 8.1.34 (End of Life)	`X-Powered-By`
3	MEDIUM	Раскрытие списка пользователей через REST API	`/wp-json/wp/v2/users`
4	MEDIUM	Отсутствуют заголовки безопасности (6 шт.)	HTTP-заголовки
5	MEDIUM	Отсутствие двухфакторной аутентификации	`/wp-login.php`
6	LOW	Раскрытие версий серверного ПО	`Server`

Содержание

1. Объём и ограничения работ
2. Методика тестирования
3. Находка № 1 — Публичный журнал отладки
4. Находка № 2 — Устаревший PHP
5. Находка № 3 — Раскрытие пользователей
6. Находка № 4 — Заголовки безопасности
7. Находка № 5 — Отсутствие 2FA
8. Находка № 6 — Версии серверного ПО
9. Приоритеты исправления

1. Объём и ограничения работ

Работы проведены в отношении единственного ресурса `https://stk-96.ru` и его публичных HTTP(S)-интерфейсов. Тестирование выполнялось в щадящем режиме, без эксплуатации уязвимостей, без подбора паролей и без нагрузки на инфраструктуру.

Разрешено и выполнено	Не выполнялось
Чтение публично доступных страниц и файлов. Анализ HTTP-заголовков и TLS-сертификата. Проверка REST API на доступ без авторизации. Проверка наличия типовых служебных файлов методом HEAD. Анализ обнаруженного файла журнала отладки.	Подбор паролей, эксплуатация CVE. SQL-инъекции, XSS, RCE, SSRF. Обход аутентификации, загрузка файлов. DoS-нагрузка, сканирование портов. Социальная инженерия.

2. Методика тестирования

1. Разрешение DNS, проверка доступности TCP/80 и TCP/443.
2. Анализ HTTP-заголовков, TLS-сертификата, политики редиректов.
3. Проверка наличия защитных заголовков безопасности.
4. Идентификация CMS и её компонентов по публичным артефактам.
5. Проверка эндпоинтов REST API на доступ без аутентификации.
6. Проверка доступности служебных и потенциально чувствительных файлов.
7. Анализ содержимого обнаруженного файла журнала отладки.

Использованный инструментарий

- PowerShell 5.1 — отправка HTTP(S)-запросов, анализ заголовков.
- Resolve-DnsName — разрешение доменных имён.
- Test-NetConnection — проверка TCP-портов 80 и 443.
- Invoke-WebRequest / Invoke-RestMethod — чтение публичных ресурсов.

3. Находка № 1 — Публично доступный журнал отладки WordPress

Уровень риска:	CRITICAL
Адрес:	`https://stk-96.ru/wp-content/debug.log`
Классификация:	CWE-532, CWE-219
Аутентификация:	не требуется

Описание

По указанному адресу без аутентификации доступен файл журнала отладки WordPress. Файл активно пополняется с 1 сентября 2026 года, на момент проверки его размер превышает 1,1 МБ. Содержимое включает полные серверные пути, названия внутренних файлов ядра CMS, а также сообщения об ошибках уровня `PHP Deprecated`, `PHP Warning` и `PHP Fatal error`.

> **Почему это критично.** Публичный журнал отладки — готовый источник разведданных: структура каталогов, версия ядра, имена пользователей и фрагменты SQL-запросов. На практике такие файлы используются как первый шаг подготовки точечной атаки.

Воспроизведение

Запрос к файлу выполняется обычным GET, без специальных заголовков.

```
PS> Invoke-WebRequest 'https://stk-96.ru/wp-content/debug.log' -UseBasicParsing -TimeoutSec 10
Status:        200
Content-Type:  text/plain
Length:        1131997
```

ФРАГМЕНТ ОТВЕТА — ПЕРВЫЕ СТРОКИ ЖУРНАЛА

```
[01-Sep-2026 13:26:45 UTC] PHP Deprecated:  preg_replace(): Passing null to parameter #3
($subject)
of type array|string is deprecated in /home/s/stk96/stk96.beget.tech/public_html/wp-
includes/kses.php on line 2025
```

ЧУВСТВИТЕЛЬНЫЕ ЭЛЕМЕНТЫ В ОТВЕТЕ

- `/home/s/stk96/stk96.beget.tech/public_html/` — полный серверный путь и имя хостинг-аккаунта;
- `wp-includes/kses.php` — точное расположение файла ядра, раскрывает версию CMS;
- `line 2025` — номер строки, позволяет сопоставить код с конкретной версией ядра.

Влияние

- Раскрытие структуры файловой системы и имени хостинг-аккаунта.

- Возможное присутствие в файле логина администратора и путей к плагинам.
- Возможное присутствие фрагментов SQL-запросов и токенов.
- Облегчение эксплуатации других, ещё не выявленных уязвимостей (LFI, RFI).
- Файл продолжает пополняться — утечка носит постоянный характер.

Рекомендации

1. Отключить режим отладки на production в `wp-config.php`:

```
define('WP_DEBUG', false);
define('WP_DEBUG_LOG', false);
define('WP_DEBUG_DISPLAY', false);
define('SCRIPT_DEBUG', false);
```

2. Удалить существующий файл `wp-content/debug.log`.
3. Закрыть на уровне nginx доступ к `*.log` в каталоге `/wp-content/`:

```
location ~* /wp-content/.*\.log$ { deny all; return 403; }
```

4. Настроить ротацию журналов и вынести их за пределы web-root.

4. Находка № 2 — Устаревшая версия PHP

Уровень риска:	HIGH
Компонент:	Интерпретатор PHP
Версия:	`8.1.34`
Классификация:	CWE-1104 (использование неподдерживаемых компонентов)

Описание

Заголовок `X-Powered-By` раскрывает точную версию PHP. Ветка 8.1 официально снята с поддержки безопасности 31 декабря 2025 года. С этого момента уязвимости, найденные в данной ветке, не получают патчей.

```
PS> (Invoke-WebRequest 'https://stk-96.ru' -Method Head -UseBasicParsing).Headers
Key           Value
Server        nginx-reuseport/1.21.1
X-Powered-By  PHP/8.1.34
```

Влияние

- Любая новая CVE в ветке PHP 8.1 останется незакрытой.
- Совместимость с новыми версиями ядра WordPress и плагинов будет ухудшаться.
- Публично раскрыта точная версия — упрощает подбор эксплойта.

Рекомендации

1. Обновить PHP до актуальной поддерживаемой ветки (8.3 или 8.4).
2. Скрыть версию: в `php.ini` установить `expose_php = Off`.
3. Перед обновлением проверить совместимость плагинов и темы.

5. Находка № 3 — Раскрытие списка пользователей через REST API

Уровень риска:	**MEDIUM**
Адрес:	`https://stk-96.ru/wp-json/wp/v2/users`
Классификация:	CWE-200 (раскрытие информации)

Описание

Эндпоинт WordPress REST API возвращает список зарегистрированных пользователей без аутентификации. В ответе раскрывается логин `admin`, что существенно упрощает подготовку атаки на форму входа.

```
PS> Invoke-RestMethod 'https://stk-96.ru/wp-json/wp/v2/users' -UseBasicParsing
id   slug    name
1    admin   admin
```

Дополнительно подтверждено: `/wp-json/wp/v2/users/me` корректно возвращает 401 — то есть защищён. Проблема именно в публичном перечислении пользователей.

Влияние

- Раскрытие действующего логина администратора.
- Упрощение брутфорса и password spraying.
- Возможность таргетированной фишинговой атаки на известного пользователя.

Рекомендации

1. Отключить публичный доступ к списку пользователей:

```
add_filter('rest_endpoints', function($endpoints) {
    unset($endpoints['/wp/v2/users']);
    unset($endpoints['/wp/v2/users/(?P<id>[\d]+)']);
    return $endpoints;
});
```

2. Переименовать учётную запись `admin` на нейтральное имя.
3. Ограничить доступ к REST API для неавторизованных запросов, если API не используется публично.

6. Находка № 4 — Отсутствуют заголовки безопасности

Уровень риска: **MEDIUM**

Классификация: CWE-693 (недостаточная защита механизма)

Описание

В ответах сервера отсутствуют шесть базовых защитных заголовков. Само по себе это не является прямой уязвимостью, но лишает браузер пользователя ряда встроенных механизмов защиты.

Заголовок	Статус	Назначение
Strict-Transport-Security	отсутствует	Защита от SSL-stripping
Content-Security-Policy	отсутствует	Снижение риска XSS
X-Frame-Options	отсутствует	Защита от clickjacking
X-Content-Type-Options	отсутствует	Защита от MIME-sniffing
Referrer-Policy	отсутствует	Контроль утечки Referer
Permissions-Policy	отсутствует	Ограничение браузерных API

Рекомендации

Добавить заголовки на уровне nginx:

```
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
add_header X-Content-Type-Options "nosniff" always;
add_header X-Frame-Options "SAMEORIGIN" always;
add_header Referrer-Policy "strict-origin-when-cross-origin" always;
add_header Permissions-Policy "geolocation=(), microphone=(), camera=()" always;
add_header Content-Security-Policy "default-src 'self'; script-src 'self' 'unsafe-inline';
style-src 'self' 'unsafe-inline'; img-src 'self' data: https:;" always;
```

CSP рекомендуется включать в режиме отчёта (`Content-Security-Policy-Report-Only`) на первой итерации, чтобы не сломать текущую вёрстку.

7. Находка № 5 — Отсутствие двухфакторной аутентификации

Уровень риска: **MEDIUM**

Адрес: `https://stk-96.ru/wp-login.php`

Описание

На форме входа не обнаружено признаков наличия второго фактора. В сочетании с раскрытым логином `admin` и устаревшей версией PHP это повышает ценность успешной компрометации учётной записи.

```
PS> $lh = (Invoke-WebRequest 'https://stk-96.ru/wp-login.php' -UseBasicParsing).Content
PS> $lh -match 'two-factor|2fa|otp|authenticator|totp'
False
```

Рекомендации

1. Установить плагин двухфакторной аутентификации (TOTP или WebAuthn).
2. Включить 2FA в первую очередь для всех пользователей с ролью administrator.
3. Дополнительно ограничить доступ к `/wp-login.php` и `/wp-admin/` по IP.

8. Находка № 6 — Раскрытие версий серверного ПО

Уровень риска: **LOW**

Классификация: CWE-200 (раскрытие информации)

Заголовки ответа содержат точные версии серверного ПО: `nginx-reuseport/1.21.1` и `PHP/8.1.34`. Само по себе это не уязвимость, но упрощает подбор подходящего эксплойта.

```
Server:        nginx-reuseport/1.21.1
X-Powered-By:  PHP/8.1.34
```

Рекомендации

- Скрыть версию nginx: `server_tokens off;` в конфигурации.
- Скрыть версию PHP: `expose_php = Off` в `php.ini`.
- Убедиться, что nginx обновлён до актуальной стабильной ветки.

9. Заключение и приоритеты исправления

Ресурс `stk-96.ru` функционирует на устаревшей версии PHP, использует CMS WordPress с публично доступным журналом отладки и не применяет базовые защитные заголовки. Совокупность этих факторов формирует устойчивую поверхность для подготовки целенаправленной атаки. Прямой эксплуатации уязвимостей в рамках согласованного объёма не производилось.

Очередь	Уровень	Действие
1	CRITICAL	Закрыть доступ к `wp-content/debug.log` и удалить файл.
2	HIGH	Обновить PHP до актуальной поддерживаемой версии.
3	**MEDIUM**	Отключить публичное перечисление пользователей через REST API.
4	**MEDIUM**	Добавить заголовки безопасности и включить HSTS.
5	**MEDIUM**	Включить двухфакторную аутентификацию для администраторов.
6	LOW	Скрыть версии nginx и PHP.

Что осталось за рамками работ

В рамках согласованного объёма не проводилось тестирование бизнес-логики, параметров GET/POST на SQL-инъекции и XSS, загрузки файлов, а также аудит сторонних плагинов и тем. При необходимости эти направления могут быть согласованы отдельно.

Повторная проверка

После устранения находок рекомендуется повторная проверка в объёме, аналогичном настоящему отчёту, для подтверждения корректности исправлений.

Отчёт подготовлен для внутреннего использования владельца.
Документ не подлежит публичному распространению без письменного согласия владельца ресурса.